

دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پیاده سازی و تحلیل سیاست های جایگزینی حافظه نهان در ChampSim

تحلیل و بررسی سیاست های جایگزینی پایه و پیشرفته و مقایسه آنها با یکدیگر

گروه ۱۳

امیررضا دولتی ۴۰۳۱۰۵۲۲۲
سینا فعالیت ۴۰۳۱۰۶۴۲۱

امیر محمد محقق ۴۰۳۱۰۶۶۱۴۶
سپهر کلانکی ۴۰۳۱۰۶۵۱۴۹

استاد درس: استاد اسدی
درس: معماری کامپیوتر

تابستان ۱۴۰۴

فهرست مطالب

۱	مقدمه	۱
۲	توضیح Replacement Policy ها	۲
۲-۱	یک نکته ی مشترک بین چند policy: نوع دسترسی access_type::WRITE	۲
۲-۲	توابع مهم برای هر سیاست	۲
۲-۳	LRU	۲
۲-۳-۱	قطعه ۱ - ساختار داده ی کمکی و سازنده	۳
۲-۳-۲	قطعه ۲ - انتخاب بلوک قربانی	۳
۲-۳-۳	قطعه ۳ - درج و به روزرسانی	۳
۲-۴	FIFO	۴
۲-۴-۱	قطعه ۱ - ساختار داده	۴
۲-۴-۲	قطعه ۲ - انتخاب قربانی و ثبت زمان ورود	۵
۲-۴-۳	قطعه ۳ - تابع خالی به روزرسانی	۵
۲-۵	MRU	۵
۲-۵-۱	قطعه ۱ - تنها تفاوت با LRU	۶
۲-۶	Random	۶
۲-۶-۱	قطعه ۱ - مولد شبه تصادفی و بازه ی انتخاب	۶
۲-۶-۲	قطعه ۲ - انتخاب قربانی و نبود توابع دیگر	۷
۲-۷	SRRIP	۷
۲-۷-۱	قطعه ۱ - helper هر set و مقداردهی اولیه	۷
۲-۷-۲	قطعه ۲ - انتخاب قربانی و افزایش گروهی	۸
۲-۷-۳	قطعه ۳ - به روزرسانی مشترک برای hit و fill	۸
۲-۸	BIP	۹
۲-۸-۱	قطعه ۱ - مولد تصادفی و پارامتر ϵ	۹
۲-۸-۲	قطعه ۲ - انتخاب قربانی (کاملاً مثل LRU)	۱۰
۲-۸-۳	قطعه ۳ - درج دو حالت	۱۰
۲-۹	DRRIP	۱۱
۲-۹-۱	قطعه ۱ - ثابت ها و شمارنده ی PSEL	۱۲
۲-۹-۲	قطعه ۲ - دو sub-policy درج	۱۳
۲-۹-۳	قطعه ۳ - انتخاب sub-policy در زمان درج	۱۳
۲-۹-۴	قطعه ۴ - hit و انتخاب قربانی	۱۴
۲-۱۰	SHiP	۱۴
۲-۱۰-۱	قطعه ۱ - ساختار sampler و جدول SHCT	۱۶
۲-۱۰-۲	قطعه ۲ - آموزش جدول از روی sampler	۱۶
۲-۱۰-۳	قطعه ۳ - استفاده از پیش بینی در زمان درج	۱۸
۲-۱۰-۴	قطعه ۴ - انتخاب قربانی	۱۸
۳	مقایسه ی سیاست ها روی ردپا ها	۱۹
۳-۱	سوال ۱: عملکرد میانگین هر سیاست	۱۹
۳-۲	سوال ۲: آیا یک سیاست ثابت در همه ی ردپا ها بهترین است؟	۱۹
۳-۳	سوال ۳: مقایسه ی سیاست های پیشرفته با LRU	۱۹
۳-۴	سوال ۴: کجا LRU و FIFO کفایت می کنند؟	۱۹
۳-۵	سوال ۵: آیا کاهش Miss Rate همیشه به افزایش IPC منجر می شود؟	۱۹
۳-۶	سوال ۶: رفتار سیاست های تطبیقی	۲۰
۳-۷	نمودار ها	۲۰
۳-۷-۱	مقایسه بر حسب سیاست، برای هر سه اندازه ی کش	۲۰
۳-۷-۲	مقایسه بر حسب تعداد set	۲۱
۴	توضیح Trace ها	۲۳
۵	تحلیل پیچیدگی و هزینه ی سخت افزاری هر Policy	۲۴

۲۵	۵-۱ جدول مقایسه ی کلی
۲۵	۵-۲ بررسی سیاست به سیاست
۲۵	۵-۲-۱ Random
۲۵	۵-۲-۲ FIFO
۲۵	۵-۲-۳ LRU
۲۶	۵-۲-۴ MRU
۲۶	۵-۲-۵ SRRIP
۲۶	۵-۲-۶ BIP
۲۶	۵-۲-۷ DRRIP
۲۶	۵-۲-۸ SHiP
۲۶	۵-۳ جمع بندی: کارایی در برابر سادگی
۲۸	۶ نحوه ی اجرا و خروجی گرفتن پروژه
۲۸	۶-۱ اسکریپت make_bins.sh – ساخت باینری ها
۲۸	۶-۲ اسکریپت run_all.sh – اجرای شبیه سازی ها
۲۸	۶-۳ اسکریپت ex_info.sh – استخراج نتایج
۲۹	مراجع

بخش ۱ مقدمه

مسئله‌ی اصلی که در این پروژه باید بررسی می‌شود، مطالعه و مقایسه‌ی Replacement Policy های مختلف Cache در لایه‌ی آخر (LLC) است. سیاست‌های بررسی شده در این گزارش به شرح زیرند:

مخفف	نام کامل	مرجع
LRU	Least Recently Used	ویکی‌پدیا [1]
MRU	Most Recently Used	ویکی‌پدیا [1]
FIFO	First In First Out	ویکی‌پدیا [1]
Random	Random Replacement	ویکی‌پدیا [1]
SRRIP	Static Re-Reference Interval Prediction	مقاله‌ی [2] RRIP
DRRIP	Dynamic Re-Reference Interval Prediction	مقاله‌ی [2] RRIP
BIP	Bimodal Insertion Policy	مقاله‌ی [3] DIP
SHiP	Signature-based Hit Predictor	مقاله‌ی [4] SHiP

جدول ۱-۱: سیاست‌های جایگزینی بررسی شده در این پروژه

شبیه‌سازی‌ها همگی با شبیه‌ساز [5] ChampSim انجام شده‌اند و ردپاهای استفاده شده را از مخزن ردپای آماده‌ی ChampSim دانلود کرده‌ایم [6].

یادداشت

برای بخش امتیازی ما تغییر تعداد set ها را انتخاب کردیم و بین مقادیر ۱۰۲۴ و ۲۰۴۸ و ۴۰۹۶ تغییرشان دادیم. تحلیل‌های خارج از حالت امتیازی روی ۲۰۴۸ انجام شده‌اند.

یادداشت

تعداد دستورات برای بخش warmup برابر ۱۲/۵ میلیون و برای بخش شبیه‌سازی برابر ۵۰ میلیون است.

توضیح Replacement Policy ها

برای تمام Replacement Policy ها یک دور روش آنها را به طور کامل توضیح می دهیم که گزارش کامل باشد. نکات ناقص رو نیز باید اضافه بکنیم.

۲-۱ یک نکته ی مشترک بین چند policy: نوع دسترسی access_type::WRITE
در چند تا از سیاست های، یک شرط روی access_type می بینیم که دسترسی های نوع WRITE را جدا می کند. این نوع دسترسی ها مربوط به writeback می شود. نکته این است که این نوشتن نشانه ی استفاده ی پردازنده از داده در همان لحظه نیست، بلکه صرفا نتیجه ی replacement در سطح بالاتر است. اگر writeback ها را مثل دسترسی عادی حساب کنیم و سیاست ها را روی آنها نیز اجرا کنیم، دو مشکل پیش می آید: اول اینکه بلوک هایی که فقط نوشته شده اند بی دلیل تازه به حساب می آیند و در کتس ماندگار می شوند و دوم اینکه در سیاست های پویا مثل DRRIP آمار تصمیم گیری منحرف می شود چون این دسترسی ها رفتار واقعی برنامه را نشان نمی دهند. به همین دلیل در سیاست های ما یا اثر writeback نادیده گرفته می شود یا بلوک با یک مقدار امن و غیر مخرب پر می شود.

۲-۲ توابع مهم برای هر سیاست

- تابع find_victim: وقتی miss رخ داد، شماره ی way قربانی را برمی گرداند.
- تابع replacement_cache_fill: بعد از اینکه بلوک جدید داخل way قرار گرفت صدا زده می شود (منطق fill).
- تابع update_replacement_state: در هر دسترسی صدا زده می شود و پارامتر hit مشخص می کند دسترسی hit بوده یا نه.

نکته ی نهایی این نیز که بیشتر تنظیمات کلی و global به ازای هر CPU Core است، مثلا برای شمارنده ها و ...

۲-۳ LRU

منطق انتخاب بلوک قربانی:

بین تمام بلوک های موجود در یک set، بلوکی حذف می شود که بیشترین زمان بدون استفاده مانده است، به همین دلیل بلوک های که دسترسی بیشتری در طول زمان دارند کمتر حذف می شوند.

نحوه بروزرسانی اطلاعات کمکی:

هر بلوک یک مقدار زمان دسترسی نگه می دارد که نشان می دهد نسبت به بقیه بلوک ها داخل همان set کی مورد دسترسی واقع شده است. مقدار ه یعنی اینکه نسبت به بقیه قدیمی تر استفاده شده است، و هرچه مقدار آن بزرگتر باشد یعنی اینکه نسبت به بقیه جدید تر مورد دسترسی واقع شده است، و جدیدترین دسترسی این set است. هر دسترسی با توجه به Miss یا Hit شدن مقادیر را تغییر می دهیم که ترتیب دسترسی حفظ شود.

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

در هنگام hit برچسب زمان همان بلوک برابر مقدار فعلی شمارنده ی cycle می شود و بعد cycle یکی زیاد می شود، یعنی آن بلوک تبدیل به جدیدترین بلوک (MRU) در set خودش می شود و برچسب بقیه ی بلوک ها دست نخورده باقی می ماند، پس به صورت نسبی همه ی آنها یک واحد زمانی قدیمی تر می شوند. تنها استثنا دسترسی های نوع WRITE است که مطابق توضیح ابتدای بخش نادیده گرفته می شوند و برچسب زمان را عوض نمی کنند.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

بعد از اینکه بلوک قربانی خارج شد و بلوک جدید در همان way نشست، تابع replacement_cache_fill برچسب زمان آن way را برابر cycle قرار می دهد و شمارنده را زیاد می کند. پس بلوک تازه درج شده در موقعیت MRU قرار می گیرد و امن ترین بلوک set است، یعنی تا وقتی که تمام بلوک های دیگر یک بار خارج نشوند نوبت به آن نمی رسد. هیچ اطلاعات دیگری در set بروزرسانی نمی شود.

شرایط عملکرد مناسب و نامناسب:

سیاست LRU وقتی خوب کار می کند که برنامه از داده هایی که استفاده می کند به تعداد بالاتری نیز مجدد استفاده بکند.

در مقابل، LRU در حالت های scan های خطی یا حلقه ای که در cache جا نشوند بد عمل می کند، چون به ترتیب داده ها یکبار مصرف می شوند و در دومی تا به سر آرایه برسیم آن داده ها دور ریخته شده اند و عملا فقط cache را کثیف کردیم.

نکات کد:

۲-۳-۱ - ساختار داده‌ی کمکی و سازنده

```
1 class lru : public champsim::modules::replacement
2 {
3     long NUM_WAY;
4     std::vector<uint64_t> last_used_cycles;
5     uint64_t cycle = 0;
6     ...
7 };
8
9 lru::lru(CACHE* cache) : lru(cache, cache->NUM_SET, cache->NUM_WAY) {}
10
11 lru::lru(CACHE* cache, long sets, long ways)
12     : replacement(cache), NUM_WAY(ways),
13     last_used_cycles(static_cast<std::size_t>(sets * ways), 0) {}
```

اطلاعات اضافه در یک بردار یک-بعدي به طول $sets * ways$ نگه داری می شود و آدرس هر بلوک با فرمول $set * NUM_WAY + way$ به دست می آید.

۲-۳-۲ - انتخاب بلوک قربانی

```
1 long lru::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
   set, ...)
2 {
3     auto begin = std::next(std::begin(last_used_cycles), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     // Find the way whose last use cycle is most distant
7     auto victim = std::min_element(begin, end);
8     assert(begin <= victim);
9     assert(victim < end);
10    return std::distance(begin, victim);
11 }
```

ابتدا با `begin` و `end` بازه‌ی مربوط به همان `set` جدا می شود و بعد `min_element` کوچک ترین برچسب زمان را پیدا می کند. چون برچسب بزرگ تر یعنی دسترسی جدیدتر. کوچک ترین مقدار دقیقاً یعنی بلوکی که از همه دیرتر استفاده شده است.

اگر چند بلوک مقدار برابر داشته باشند (مثلاً چند `way` هنوز دست نخورده و برابر ه باشند)، `min_element` اولین آنها را برمی گرداند، پس ترتیب پر شدن `set` از `way` کوچک به بزرگ است.

۲-۳-۳ - قطعه ۳ - درج و به روزرسانی

```
1 void lru::replacement_cache_fill(uint32_t triggering_cpu, long set, long
   way, ...)
2 {
3     // Mark the way as being used on the current cycle
```

```

4     last_used_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
5 }
6
7 void lru::update_replacement_state(..., uint8_t hit)
8 {
9     // Mark the way as being used on the current cycle
10    if (hit && access_type{type} != access_type::WRITE) // Skip this for writeback hits
11        last_used_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
12 }

```

در زمان درج بلوک جدید در موقعیت MRU قرار می‌گیرد و کم‌خطرترین بلوک set است.

FIFO ۲-۴

منطق انتخاب بلوک قربانی:

بین تمام بلوک‌های موجود در set، بلوکی که زودتر از همه وارد شده است باید خارج شود (منطق First In First Out). اینکه بلوک اخیراً دسترسی قرار گرفته یا نه تأثیری در تصمیم‌گیری ندارد. عملاً منطق صف دارد. نحوه بروزرسانی اطلاعات کمکی:

یک ترتیب ورود برای بلوک‌هایی که به set وارد می‌شوند ذخیره می‌کند. موقع hit چیزی را تغییر نمی‌دهد ولی موقع miss ترتیب عوض می‌شود (البته بدون آپدیت کردن مقدار بقیه چون زمان اضافه شدن اهمیت دارد).

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

هیچ بروزرسانی‌ای انجام نمی‌شود و تابع update_replacement_state کاملاً خالی است.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

بعد از درج بلوک جدید، برچسب آن way برابر مقدار فعلی cycle قرار می‌گیرد و شمارنده یکی زیاد می‌شود، یعنی بلوک جدید به انتهای صف اضافه می‌شود. برچسب بقیه ی بلوک‌ها دست نمی‌خورد چون ترتیب ورود آنها ثابت است و ربطی به بلوک جدید ندارد.

شرایط عملکرد مناسب و نامناسب:

سیاست FIFO وقتی خوب عمل می‌کند که ترتیب ورودی داده‌ها با ترتیب استفاده از آنها هماهنگ باشد.

در جاهایی بد عمل می‌کند که بلوکی زود وارد cache شود ولی چون صرفاً زود وارد شده ممکن است با اینکه در آینده ی نزدیک خیلی استفاده می‌شود دور ریخته شود، و هوشمندی این را ندارد که اگر یک بلوک زیاد استفاده می‌شود نباید سریع بیرون بیندازدش.

نکات کد:

۲-۴-۱ قطعه ۱ – ساختار داده

```

1 class fifo : public champsim::modules::replacement
2 {
3     long NUM_WAY;
4     std::vector<uint64_t> insertion_cycles;
5     uint64_t cycle = 0;
6     ...
7 };

```

C++

ساختار داده دقیقاً مثل LRU است و تنها چیزی که فرق می‌کند مفهوم عدد کمکی است، که اینجا مفهوم زمان وارد شدن بلوک به set را می‌دهد. cycle در اینجا یک شمارنده کلی است که برای زمان اضافه شدن استفاده می‌شود و بیانگر شمارشگر زمان است که در هنگام هر fill افزایش پیدا می‌کند.

۲-۴-۲ - انتخاب قربانی و ثبت زمان ورود

```

1  long fifo::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
   set, ...)
2  {
3      auto begin = std::next(std::begin(insertion_cycles), set * NUM_WAY);
4      auto end = std::next(begin, NUM_WAY);
5
6      // Evict the oldest inserted block
7      auto victim = std::min_element(begin, end);
8      return std::distance(begin, victim);
9  }
10
11 void fifo::replacement_cache_fill(uint32_t triggering_cpu, long set, long way, ...)
12 {
13     // Record insertion time
14     insertion_cycles.at((std::size_t)(set * NUM_WAY + way)) = cycle++;
15 }

```

بدون اینکه صف واقعی داشته باشیم، با یک برچسب زمان ورود همان رفتار صف را می‌سازیم، کمترین مقدار یعنی قدیمی‌ترین ورود که همان بلوک از صف خارج می‌شود.

۲-۴-۳ - تابع خالی به روزرسانی

```

1  void fifo::update_replacement_state(..., uint8_t hit)
2  {
3  }

```

تابع update این policy خالی است چون طبق توضیحات در FIFO لازم نیست کاری انجام بدهیم.

۲-۵ MRU

منطق انتخاب بلوک قربانی:

کاملاً مخالف LRU عمل می‌کند ولی منطق مشابهی دارد به این صورت که بلوکی در set خارج می‌شود که نسبت به بقیه اخیراً دسترسی داشته است.

نحوه بروزرسانی اطلاعات کمکی:

مشابه LRU و نکته ای اضافه ای ندارد.

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

دقیقاً مثل LRU، برچسب زمان بلوکی که hit شده برابر cycle می‌شود و شمارنده زیاد می‌شود (و باز هم دسترسی های WRITE نادیده گرفته می‌شوند). ولی معنای این کار برعکس LRU است.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

بلوک تازه fill شده بزرگترین برچسب زمان set را می‌گیرد، پس بلافاصله پرخطرترین بلوک برای خروج می‌شود.

شرایط عملکرد مناسب و نامناسب:

می‌شود گفت نقطه ی مقابل LRU است و هر جا آن بد عمل می‌کند این خوب عمل می‌کند و هر جا آن خوب عمل می‌کند این بد عمل می‌کند. صرفاً اینکه برای ذخیره ی استفاده ی مکرر از داده های جدید خیلی بد عمل می‌کند. نکات کد:

۲-۵-۱ - تنها تفاوت با LRU

```

1 long mru::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     auto begin = std::next(std::begin(last_used_cycles), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     auto victim = std::max_element(begin, end); //MRU!!!
7     return std::distance(begin, victim);
8 }

```

کل تفاوت پیاده سازی با LRU استفاده از max_element به جای min_element است. به همین دلیل بلوکی که اخیراً به set اضافه شده است قربانی است. بقیه ی کد ها با LRU کاملاً یکسان هستند.

۲-۶ Random

منطق انتخاب بلوک قربانی:

در هر set به صورت رندوم یک بلوک را انتخاب می کنیم.

نحوه بروزرسانی اطلاعات کمکی:

اطلاعات اضافه ای برای ذخیره لازم نداریم.

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

هیچ چیزی بروزرسانی نمی شود و اصلاً تابع update_replacement_state را override نکرده ایم. چون هیچ حالتی برای نگه داشتن وجود ندارد. تنها چیزی که با هر انتخاب قربانی تغییر می کند وضعیت داخلی generator شبه تصادفی است.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

اینجا هم هیچ کاری انجام نمی شود و replacement_cache_fill خالی است.

شرایط عملکرد مناسب و نامناسب:

سیاست Random وقتی نسبتاً خوب عمل می کند که پترن دسترسی برنامه بدون منطق مشخص و رندوم باشد. صرفاً یک روش نامنظم در برابر نامنظم است که لزوماً بهترین نتیجه را نمی دهد و یک روش ساده ی نسبتاً معمولی است.

در برنامه هایی که پترن مشخص مثل scan خطی و استفاده ی مکرر از داده ها دارند اصلاً خوب عمل نمی کند چون یک رفتار نامنظم در برابر یک رفتار منظم است.

نکات کد:

۲-۶-۱ - مولد شبه تصادفی و بازه ی انتخاب

```

1 struct random : public champsim::modules::replacement {
2     std::mt19937_64 rng{};
3     std::uniform_int_distribution<long> dist;
4     ...
5 };
6
7 random::random(CACHE* cache) : random(cache, cache->NUM_WAY) {}
8
9 random::random(CACHE* cache, long ways) : replacement(cache), dist(0, ways - 1) {}

```

رندوم روی بازه ی $[0, \text{ways}-1]$ یک عدد انتخاب می کنیم که بیانگر آن بلوکی است که باید حذف شود.

۲-۴-۲ - انتخاب قربانی و نبود توابع دیگر

```
1 long random::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     return dist(rng);
4 }
```

در سیاست random کل خطی که منطق این سیاست را پیاده سازی می کند همین است و توابع دیگر نیاز به override کردن ندارند چون این سیاست منطق هندل کردن حالت ندارد.

SRRIP ۲-U

به ازای هر بلوک در set یک عدد به نام RRPV (Re-Reference Prediction Value) ذخیره می شود که معمولا ۲ بیت است که ۴ مقدار می تواند به خود بگیرد. در صورتی که ۰ باشد یعنی اینکه احتمالا به زودی استفاده می شود و در صورتی که ۳ باشد احتمالا خیلی در آینده ی دور استفاده می شود و در آینده ی نزدیک احتمالا استفاده نمی شود. منطق انتخاب بلوک قربانی:

در روش SRRIP (Static Re-Reference Interval Prediction) کل منطق بر این است که در یک set بلوک هایی که خیلی استفاده می شوند پایدار می شوند و در صورتی که استفاده ی یک بلوک کم شود احتمالا بلوک قربانی خواهد بود. برای پیدا کردن بلوک قربانی هم بلوکی که RRPV برابر ۳ (بیشترین مقدار) را دارد خارج می شود. و در صورتی که هیچ بلوکی مقدار ۳ نداشت، مقدار RRPV تمام بلوک ها را زیاد می کنیم تا یکی به ۳ برسد، و این فرایند تا شرط برقرار شود ادامه پیدا می کند. (بخشی از بروزرسانی هم انجام می دهد) نحوه بروزرسانی اطلاعات کمکی:

در صورتی که بلوک جدید اضافه شود، یک بلوک با RRPV تقریبا میانه مثل MAX_RRPV-1 اضافه می کند. در هنگام hit مقدار RRPV آن بلوک به ۰ ریست می شود. مقدار بروزرسانی هنگام خارج شدن یک بلوک هم داریم که بالا توضیح دادیم.

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

مقدار RRPV بلوکی که hit شده به ۰ ریست می شود، یعنی پیش بینی می کنیم که خیلی زود دوباره استفاده می شود و آن را به امن ترین حالت می بریم.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

ابتدا در خود find_victim و قبل از fill کردن، اگر بیشترین RRPV موجود در set کمتر از maxRRPV باشد به همه ی بلوک های آن set به اندازه ی $\text{diff} = \text{maxRRPV} - \text{max}$ اضافه می شود تا قربانی مشخص شود. بعد از آن برای بلوک تازه اضافه شده مقدار $\text{maxRRPV} - 1$ می دهیم.

شرایط عملکرد مناسب و نامناسب:

صرفا یک LRU پیشرفته تر است، پس یکم هوشمندانه تر از LRU عمل می کند و حالت های خوب آن مثل scan های خطی و ... است. که چون کمی هوشمندانه تر عمل می کند از LRU هم بهتر عمل می کند. حالت های بد آن تقریبا مشابه LRU وقتی است که در اسکن حلقه ای cache پر شود و قبل از اینکه به سر حلقه برسیم داده های اولیه دور ریخته شوند و مجدد از اول باید دریافت شوند که همه چیز هدر می رود. نکات کد:

۲-U-۱ - helper هر set و مقداری اولیه

```
1 struct srrip_set_helper {
2     using rrpv_type = int;
3     static constexpr rrpv_type maxRRPV = 3;
4
5     std::vector<rrpv_type> rrpv_values;
6     rrpv_type& get_rrpv(long way);
7 }
```

```

8     long victim();
9     void update(long way, bool hit);
10 };
11
12 srrip_set_helper::srrip_set_helper(long ways)
13     : rrpv_values(static_cast<std::size_t>(ways), maxRRPV) {}
14
15 srrip::srrip(CACHE* cache, long sets_, long ways_) : replacement(cache)
16 {
17     std::generate_n(std::back_inserter(sets), sets_,
18                     [ways = ways_] { return srrip_set_helper{ways}; });
19 }

```

برای داده‌های کمی یک شی `srrip_set_helper` تعریف می‌کنیم که داده‌های مورد نیاز را به صورت گروهی نگه دارد.

مقدار اولیه‌ی همه‌ی `Way` ها `maxRRPV` است، یعنی کش خالی طوری رفتار می‌کند که انگار همه‌ی بلوک‌ها در مرز خروج هستند. با این حرکت `Way` های خالی بدون هیچ بررسی اضافه‌ای اول از همه انتخاب می‌شوند. در اینجا ما `maxRRPV` را مساوی ۳ قرار دادیم که بیانگر همان ۲ بیت برای این داده است.

۲-۱-۲ - انتخاب قربانی و افزایش گروهی

```

1 long srrip_set_helper::victim()
2 {
3     // Find the maximum RRPV
4     auto victim = std::max_element(std::begin(rrpv_values), std::end(rrpv_values));
5
6     // If the maximum element has RRPV less than the maximum, increment everything to
7     // the maximum
8     std::transform(std::cbegin(rrpv_values), std::cend(rrpv_values),
9                   std::begin(rrpv_values),
10                    [diff = maxRRPV - *victim](auto x) { return x + diff; });
11
12     // Return the way index
13     return std::distance(std::begin(rrpv_values), victim);
14 }

```

همان توصیف حلقه شگلی که در توضیح سیاست داشتیم اینجا به صورت یک عبارت جبری ساده به صورت $\text{diff} = \text{maxRRPV} - \text{max}$ است که به همه اضافه می‌شود. عملاً همان کار حلقه را می‌کند و این دو روش هم ارزند ولی این بار محاسباتی کمتری دارد.

اگر از قبل بلوکی با مقدار بیشینه وجود داشته باشد، diff برابر صفر می‌شود و `transform` عملاً هیچ تغییری نمی‌دهد. یعنی همان حالت عادی که مستقیماً قربانی پیدا می‌شود.

نکته‌ی دیگر این است که این افزایش فقط در مسیر `miss` اتفاق می‌افتد.

۲-۱-۳ - به روزرسانی مشترک برای `hit` و `fill`

```

1 void srrip_set_helper::update(long way, bool hit) { get_rrpv(way) = hit ?
2   0 : (maxRRPV - 1); }

```

```

2
3 void srrip::update_replacement_state(..., uint8_t hit)
4 {
5     sets.at(static_cast<std::size_t>(set)).update(way, hit);
6 }

```

کد replacement_cache_fill بازنویسی نشده است و هر دو حالت hit و fill از همین یک تابع استفاده می کنند که برای اینکه بین hit و fill تشخیص دهیم یک bool hit داریم که بین این دو انتخاب کند. در صورتی که hit بشود مقدار به ۰ ریست می شود و در صورتی که fill باشد مقدار جدید برابر 1-maxRRPV خواهد بود.

BIP ۲-N

منطق انتخاب بلوک قربانی:

روش BIP (Bimodal Insertion Policy) عملاً روی روش LRU سوار شده است که منطق خارج کردن بلوک از set آن دقیقاً مشابه LRU است. صرفاً نحوه ی هندل کردن ترتیب بلوک ها بر اساس دسترسی اخیر آنها کمی متفاوت است.

نحوه بروزرسانی اطلاعات کمکی:

در روش LRU در هنگامی که بلوک جدید اضافه می شود، نسبت به بقیه در ترتیب دسترسی بالاتر بود. ولی در این روش هنگامی که بلوک جدید اضافه می شود با یک احتمال مشخص زیادی در ترتیب دسترسی پایینترین خواهد بود (Least Recently Used) که یعنی در خطر خارج شدن است (بر خلاف روش عادی که بلوکی که جدید اضافه می شد کم خطر ترین برای خارج شدن بود) و با یک احتمال متمم مشخص دیگری در ترتیب دسترسی نسبت به بقیه بالاترین خواهد بود (مثل حالت عادی روش LRU).

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

در هنگام hit دقیقاً مثل LRU عمل می کنیم و برچسب زمان بلوک برابر cycle می شود و شمارنده زیاد می شود (باز هم به جز دسترسی های WRITE).

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

بعد از fill کردن، با احتمال $1/32$ برچسب زمان برابر cycle می شود (درج در موقعیت MRU، مثل LRU) و با احتمال $31/32$ برچسب برابر ۰ قرار می گیرد که یعنی درج در موقعیت LRU. پس به احتمال زیاد بلوک جدید در اولین miss بعدی همان set دوباره خارج می شود و محتوای قدیمی set تقریباً دست نخورده باقی می ماند. آن احتمال کوچک $1/32$ هم برای این است که اگر working set برنامه عوض شد، کش بتواند به تدریج محتوای جدید را بپذیرد و برای همیشه روی داده های قدیمی قفل نشود.

شرایط عملکرد مناسب و نامناسب:

در این روش هم مشکل scan های حلقه ای که از سایز cache بزرگ تر می شوند تا حدی رفع می شود و هم اسکن های خطی که داده های بردنخور اضافه می شود و فقط یکبار استفاده می شوند، چون یک روش میانی بین LRU و MRU است این امکان به وجود می آید. چون با یک احتمال کم بعضی داده ها را نگه می دارد و خیلی ها را دور می اندازد، باعث نمی شود که همه چیز یادمان برود و خیلی چیز ها نگه داشته می شوند؛ در اسکن های خطی هم داده های یکبار مصرف اکثراً دور ریخته می شوند و فقط بخش کمی از آنها نگه داشته می شود.

در جایی بد عمل می کند که کل working set داخل cache جا شود و صرفاً بخاطر بدبینی این سیاست خیلی از داده ها بیرون ریخته شوند.

نکات کد:

۲-N-۱ قطعه ۱ – مولد تصادفی و پارامتر ε

```

1 class bip : public champsim::modules::replacement
2 {
3     long NUM_WAY;
4     std::vector<uint64_t> last_used_cycles;
5     uint64_t cycle = 0;

```

C++

```

6      std::mt19937 rng;
7      std::uniform_int_distribution<int> dist;
8      ...
9  };
10
11  bip::bip(CACHE* cache, long sets, long ways)
12      : replacement(cache),
13        NUM_WAY(ways),
14        last_used_cycles((std::size_t)(sets * ways), 0),
15        rng(0),
16        dist(0, 31)          // 1/32 probability
17  {
18  }
    
```

طبق توزیع $\text{dist}(0, 31)$ و شرطی که در قطعه ی سوم می آید ($== 0$) یعنی احتمال درج در موقعیت MRU دقیقاً $1/32$ و احتمال درج در موقعیت LRU برابر $31/32$ است. (این همان احتمال هایی است که در توضیح این policy شرح دادیم) این مقدار به صورت رسمی ϵ نام دارد و انتخاب آن دلخواه نیست و باید هوشمندانه باشد، ولی مقدار معمولی آن همین است. باید آنقدر کوچک باشد که برای جست و جوی خطی حافظه باعث cache pollution نشود (داده های الکی و اضافه پر نکند) و آنقدر بزرگ باشد که اگر working set برنامه عوض شد، cache بتواند محتوای جدید را کم بپذیرد.

برای seed برای رندوم هم $\text{rng}(0)$ صفر را می دهیم.

بقیه ی ساختار داده دقیقاً همان LRU است.

۲-۱-۲ - انتخاب قربانی (کاملاً مثل LRU)

```

1  long bip::find_victim(uint32_t, uint64_t, long set, ...)
2  {
3      auto begin = std::next(last_used_cycles.begin(), set * NUM_WAY);
4      auto end = std::next(begin, NUM_WAY);
5
6      auto victim = std::min_element(begin, end);
7      return std::distance(begin, victim);
8  }
    
```

کاملاً مشابه LRU برای قربانی عمل می کنیم و منطق پیشرفته تری ندارد.

۲-۱-۳ - درج دو حالت

```

1  void bip::replacement_cache_fill(uint32_t, long set, long way, ...)
2  {
3      auto idx = (std::size_t)(set * NUM_WAY + way);
4
5      // 1/32 insert at MRU
6      if (dist(rng) == 0)
7      {
8          last_used_cycles[idx] = cycle++;
8      }
    
```

```

9      }
10     else
11     {
12         // Insert as LRU
13         last_used_cycles[idx] = 0;
14     }
15 }
16
17 void bip::update_replacement_state(..., uint8_t hit)
18 {
19     if (hit && type != access_type::WRITE)
20     {
21         last_used_cycles[(std::size_t)(set * NUM_WAY + way)] = cycle++;
22     }
23 }

```

فقط منطق fill طبق توضیحی که دادیم فرق می کرد و بقیه کاملاً مشابهند.

۲-۹ DRRIP

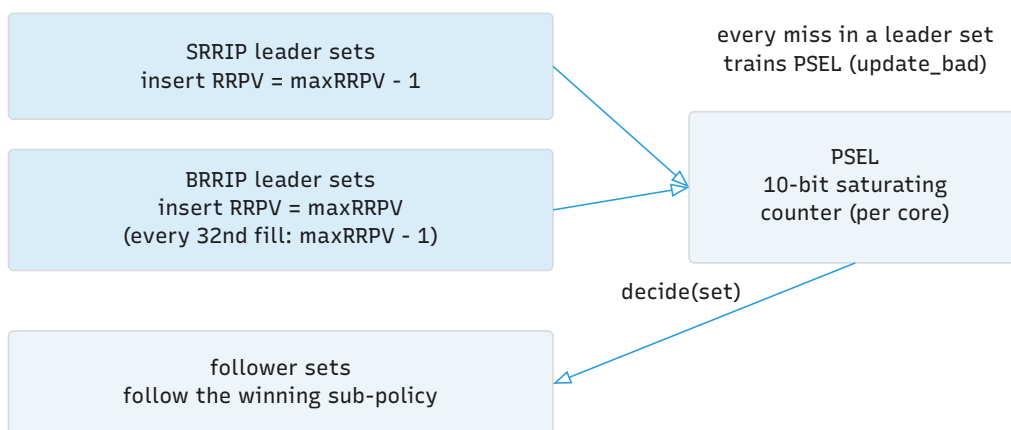
منطق انتخاب بلوک قربانی:

در روش DRRIP (Dynamic RRIP) منطق خارج کردن بلوک دقیقاً مشابه روش SRRIP است و تفاوتی ندارد.

نحوه بروزرسانی اطلاعات کمکی:

این روش هم چون یک روش تکمیلی روی SRRIP است تقریباً مشابه عمل می کند به جز قراردادی که در هنگام اضافه کردن بلوک به set دارد. به این صورت عمل می کند که دو تا sub-policy دارد که بین این دو تا به صورت پویا عوض می کند. sub-policy اول به این صورت است که برای بلوک جدیدی که اضافه می شود مثل SRRIP برای مقدار RRPV آن MAX_RRPV-1 را قرار دهیم. و sub-policy دیگر این است که مثل BIP، با احتمال خیلی زیاد (ممکن است پیاده سازی این مورد به صورت احتمالی نباشد، مثلاً در پیاده سازی ما اینطور نیست) مقدار RRPV آن را MAX_RRPV قرار دهیم (مرز خروجی)، و با احتمال خیلی کمتر MAX_RRPV-1 را قرار دهیم. در این روش اینکه کدام یک از این sub-policy ها انتخاب می شود با استفاده از روش set-dueling انجام می شود. به این صورت که ما ۳ نوع دسته داریم: brrip_leader و srrip_leader و follower. برای دسته های srrip_leader و brrip_leader به صورت ثابت به ترتیب از sub-policy های SRRIP و BRRIP استفاده می کنیم ولی برای دسته آخر که follower است با توجه به یک شمارنده PSEL انجام می شود.

هنگام hit هم فارغ از اینکه sub-policy کدام باشد مقدار RRPV به ۰ تنظیم می شود.



شکل ۲-۱: سازوکار set-dueling و نقش شمارنده ی PSEL در DRRIP

نحوه ی بروزسانی اطلاعات پس از برخورد (hit):

مستقل از اینکه set از نوع leader است یا follower و مستقل از اینکه کدام sub-policy فعال است، مقدار RRPV بلوکی که hit شده به ۰ ریست می شود. برای دسترسی های WRITE به جای ۰ مقدار 1 - maxRRPV گذاشته می شود تا writeback باعث نشود بلوکی الکی امن به حساب بیاید. نکته ی مهم دیگر این است که در مسیر hit شمارنده ی PSEL دست نمی خورد و آموزش PSEL فقط از روی miss ها انجام می شود.

نحوه ی بروزسانی اطلاعات پس از جایگزینی (fill):

اول مثل SRRIP، در find_victim مقدار RRPV همه ی بلوک های set به اندازه ی لازم زیاد می شود تا قربانی با مقدار بیشینه پیدا شود. بعد از درج بلوک جدید، تابع decide(set) مشخص می کند کدام sub-policy اجرا شود: set های leader همیشه سیاست ثابت خودشان (SRRIP یا BRRIP) و set های follower بر اساس مقدار فعلی PSEL. در حالت SRRIP مقدار 1 - maxRRPV و در حالت BRRIP مقدار maxRRPV داده می شود، به جز یک بار در هر ۳۲ درج که به صورت چرخشی مقدار 1 - maxRRPV می گیرد.

سپس برای خود PSEL داریم: در پایان هر fill تابع update_bad(set) صدا زده می شود. چون fill فقط بعد از miss اتفاق می افتد، هر miss در یک set رهبر شمارنده را به ضرر سیاست همان رهبر حرکت می دهد و بعد از مدتی PSEL به سمت سیاستی می رود که در set های نمونه miss کمتری داشته است. این تنها بخش واقعا پویای این روش است (شمارنده ی PSEL از روی set های leader یاد می گیرد و مثل دیکتاتور به set های follower غالب می کند).

شرایط عملکرد مناسب و نامناسب:

سیاست DRRIP تقریباً در همه ی شرایط خوب عمل می کند چون به صورت پویا بین دو روش جابه جا می شود. صرفاً در جاهایی ممکن است بد عمل کند که working set اینقدر کم باشد که روش فرصت یادگیری نداشته باشد و overkill باشد.

نکات کد:

۹-۲-۱ قطعه ۱ - ثابت ها و شمارنده ی PSEL

```
1 struct drrip : public champsim::modules::replacement {
2     static constexpr unsigned maxRRPV = 3;
3     static constexpr unsigned BRRIP_MAX = 32;
4     static constexpr unsigned PSEL_WIDTH = 10;
5
6     enum class set_type { follower, brrip_leader, srrip_leader };
7
8     unsigned brrip_counter;
9
10    std::vector<unsigned> rrpv;
11    std::vector<champsim::msl::dscounter<long, PSEL_WIDTH>> PSEL;
12 };
13
14 drrip::drrip(CACHE* cache)
15     : replacement(cache), NUM_SET(cache->NUM_SET), NUM_WAY(cache->NUM_WAY),
16       rrpv(static_cast<std::size_t>(NUM_SET * NUM_WAY)),
17       PSEL(NUM_CPUS, champsim::msl::dscounter<long,
18             PSEL_WIDTH>(champsim::msl::get_sample_rate(NUM_SET)))
19 { }
```

یک شمارنده ی PSEL با طول ۱۰ بیت به ازای هر هسته داریم. ۳ دسته برای set ها طبق توضیحی که دادیم داریم. و بقیه ی المان های SRRIP و المان های مشابه BIP را نیز مشاهده می کنیم.

۲-۹-۲ قطعه ۲ – دو sub-policy درج

```

1 void drrip::update_brrip(long set, long way)
2 {
3     get_rrpv(set, way) = maxRRPV;
4
5     brrip_counter++;
6     if (brrip_counter == BRRIP_MAX) {
7         brrip_counter = 0;
8         get_rrpv(set, way) = maxRRPV - 1;
9     }
10 }
11
12 void drrip::update_srrip(long set, long way) { get_rrpv(set, way) = maxRRPV - 1; }

```

در حالت BRRIP بلوک جدید در ۳۱ حالت از هر ۳۲ حالت با مقدار maxRRPV اضافه می‌شود (اینجا یک تفاوت ریز با BIP دارد و آن این است که در BIP این موضوع رندوم بود ولی اینجا به صورت چرخشی و سیکلی است و رندوم نیست، تاثیر عملکردی آن شاید ناچیز باشد و تقریباً یکسان باشند ولی از لحاظ سخت افزاری خیلی ساده تر از رندوم است)، یعنی در مرز خروج قرار می‌گیرد و اگر تا اولین جست و جوی قربانی استفاده نشود بیرون می‌رود. فقط یک بار در هر ۳۲ درج، مقدار maxRRPV - 1 داده می‌شود تا Cache بتواند به تدریج محتوای جدید را نگه دارد. تابع update_srrip دقیقاً همان درج SRRIP است و هیچ تفاوتی با آن ندارد.

۲-۹-۳ قطعه ۳ – انتخاب sub-policy در زمان درج

```

1 void drrip::replacement_cache_fill(uint32_t triggering_cpu, long set, long
2 way, ...)
3 {
4     // do not update replacement state for writebacks
5     if (access_type{type} == access_type::WRITE) {
6         get_rrpv(set, way) = maxRRPV - 1;
7         return;
8     }
9     if (PSEL[triggering_cpu].decide(set)) {
10         update_brrip(set, way);
11     } else {
12         update_srrip(set, way);
13     }
14     // cache miss, update bad
15     PSEL[triggering_cpu].update_bad(set);

```

تابع decide(set) برای set های leader همیشه سیاست فیکس شده ی خودشان را برمی گرداند و برای set های follower بر اساس مقدار فعلی PSEL تصمیم می‌گیرد.

تابع update_bad(set) در هر fill صدا زده می‌شود و چون fill فقط بعد از miss اتفاق می‌افتد، عملاً هر miss در یک رهبر، PSEL را به ضرر سیاست همان رهبر تغییر می‌دهد. در نتیجه بعد از مدتی PSEL به سمت سیاستی

می رود که در set های نمونه miss کمتری داده است. این همان بخش پویای این روش است. اینکه خود تابع update_bad چطور پیاده سازی شده است در فایل های داخلی ChampSim است و خیلی جزیی می شود.

۲-۹-۴ - hit و انتخاب قربانی

```

1 void drrip::update_replacement_state(..., uint8_t hit)
2 {
3     if (hit) {
4         if (access_type{type} == access_type::WRITE) {
5             get_rrpv(set, way) = maxRRPV - 1;
6             return;
7         }
8         get_rrpv(set, way) = 0; // DRRIP always promotes a cache line to the MRU
           position
9     }
10 }
11
12 long drrip::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long set, ...)
13 {
14     auto begin = std::next(std::begin(rrpv), set * NUM_WAY);
15     auto end = std::next(begin, NUM_WAY);
16
17     auto victim = std::max_element(begin, end);
18     if (auto rrpv_update = maxRRPV - *victim; rrpv_update != 0)
19         for (auto it = begin; it != end; ++it)
20             *it += rrpv_update;
21
22     return std::distance(begin, victim);
23 }
    
```

تابع find_victim از نظر منطقی دقیقاً همان SRRIP است و فقط به جای std::transform از یک حلقه استفاده کرده است.

در زمان برخورد، بدون توجه به اینکه کدام sub-policy فعال است، مقدار RRPV به ۰ می رسد.

SHiP ۲-۱۰

منطق انتخاب بلوک قربانی:

در روش SHiP (Signature-based Hit Predictor)، کاملاً مشابه روش SRRIP مقادیر RRPV به ازای هر بلوک داریم که دقیقاً به همان روش SRRIP بلوک هدف را خارج می کنیم.

نحوه بروزرسانی اطلاعات کمکی:

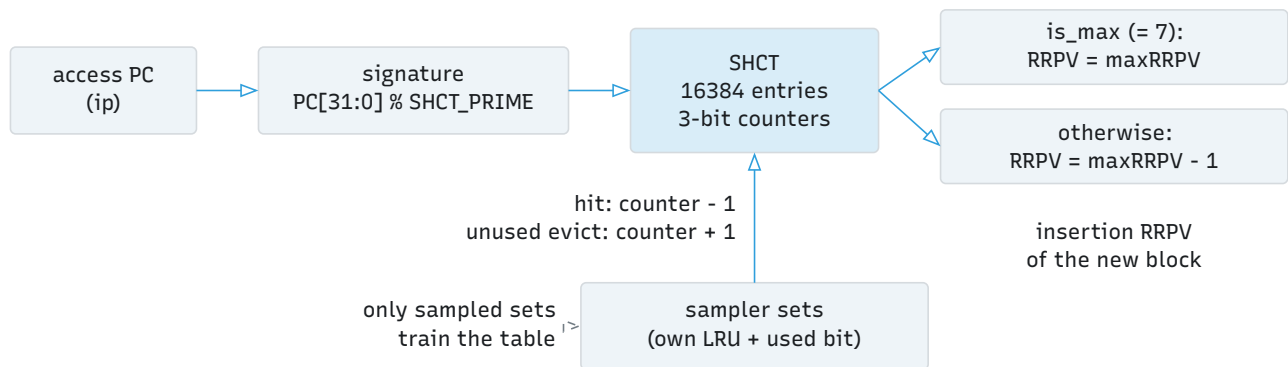
در این policy ما روی مقدار اولیه RRPV که به ازای هر بلوک نسبت می دهیم خلاقیت به خرج می دهیم و سعی می کنیم مقدار هوشمندانه ای برای آن تعیین کنیم. به این صورت که در کنار RRPV یک تگ به اسم signature هم به هر بلوک هنگام اضافه شدن به set به آن می دهیم که بر اساس ناحیه ی حافظه ای است که در آن قرار داشته است یا آدرس حافظه مربوط به آن است (PC) (کلاً قرارداد مخصوص به خود دارد، مثلاً در کد ما همان PC است). یک جدول به اسم SHCT (Signature History Counter Table) نگه می داریم که به ازای هر signature یک شمارنده دارد که در هر بار دسترسی به بلوک با آن signature خاص شمارنده مربوط به signature در جدول کم می شود. عملاً مقدار به ازای هر signature در این جدول بیان می کند که این پترن خاص از حافظه چقدر امکان مجدد

بازخواست و دسترسی دارند. هنگام اضافه کردن بلوک به set از این جدول lookup انجام می‌دهیم و مقدار مربوط به signature این بلوک را می‌خوانیم، بر اساس این مقدار RRPV را مقداردهی می‌کنیم. در صورتی که این مقدار کم باشد و یعنی احتمال بازخواست این signature زیاد باشد مقدار RRPV آن را یک چیز کم مثل ۰ یا ۱ می‌دهیم (البته اینکه در حالت خوب یک مقدار RRPV به آن داده شود در کد نیست و در اولین پیاده‌سازی های این ایده هم ظاهر نبوده و در مقالات و ورژن های بعدی که بهبود یافته است ظاهر می‌شود). و اگر مقدار جدول زیاد باشد و یعنی این signature احتمالا زیاد بازخواست نمی‌شود مقدار RRPV بالایی مثل MAX_RRPV به آن نسبت می‌دهیم.

در هنگام hit مقدار RRPV بلوک به ۰ ریست می‌شود و ورودی داخل جدول SHCT که مربوط به signature این بلوک است یکی کم می‌شود.

در هنگامی که بلوک بدون اینکه یکبار هم hit شده باشد در جدول مقدار مربوط به signature آن یکی زیاد می‌شود.

نکته ی بسیار مهم این است که این جدول اگر برای تمام set های موجود ذخیره شود سخت افزار خیلی بزرگی می‌خواهد. پس به جای اینکه ما این را برای تعداد محدودی set انجام می‌دهیم و به بقیه set ها تعمیم می‌دهیم. به این sampler می‌گوییم که چیزی شبیه شبکه عصبی که با یک سری set ها train می‌شود و روی بقیه تست می‌شود. به این شکل سخت افزار کمتری مصرف می‌شود. در جاهای مختلف کد این موضوع تکرار شده ولی چون از توضیحی که دادیم خارج نیست، فرض می‌کنیم کلا sampler را ندید می‌گیریم و کلی توضیح می‌دهیم. پس داخل کد و توضیحات بیشتر جاها منظورمان خود sampler است که در یادگیری استفاده می‌شود ولی برای ساده سازی به خود جدول اشاره کردیم.



شکل ۲-۲: مسیر پیش بینی و مسیر آموزش جدول SHCT در SHiP

نحوه ی بروزرسانی اطلاعات پس از برخورد (hit):

ابتدا در کش اصلی مقدار RRPV آن بلوک مثل SRRIP به ۰ ریست می‌شود و سپس برای train اگر set مورد نظر جزو set های sampler باشد، خانه ی متناظر در جدول SHCT پیدا می‌شود و مقدار آن یکی کم می‌شود، چون این hit ثابت می‌کند که آن signature (یعنی همان PC) داده ای می‌آورد که واقعا دوباره استفاده می‌شود. همچنین بیت used آن خانه ی sampler برابر true می‌شود تا بعدا موقع خروج، این بلوک به عنوان «بدر نخور» جریمه نشود. اگر set جزو نمونه ها نباشد هیچ train ای انجام نمی‌شود و فقط RRPV ریست می‌شود.

نحوه ی بروزرسانی اطلاعات پس از جایگزینی (fill):

ابتدا signature بلوک جدید از ۳۲ بیت پایین PC ساخته و با SHCT_PRIME % روی جدول مپ می‌شود و مقدار آن خانه خوانده می‌شود. اگر شمارنده به بیشینه رسیده باشد (is_max، یعنی مقدار u) بلوک با maxRRPV درج می‌شود که یعنی همان لحظه در مرز خروج است، وگرنه مقدار پیشفرض maxRRPV - 1 مثل SRRIP را می‌گیرد. برای دسترسی های WRITE هم بدون هیچ lookup ای همان مقدار امن maxRRPV - 1 گذاشته می‌شود.

سپس برای train وقتی در یک set نمونه یک خانه ی sampler بازنویسی می‌شود، اگر آن خانه بیت used نداشته باشد یعنی بلوک قبلی بدون حتی یک بار استفاده خارج شده است، پس شمارنده ی مربوط به signature آن یکی زیاد می‌شود. بعد خانه با آدرس و PC جدید مقداردهی و used دوباره صفر می‌شود. توجه کنیم که sampler برای خودش یک سیاست LRU مستقل با last_used دارد که هیچ ربطی به RRPV های کش اصلی ندارد.

شرایط عملکرد مناسب و نامناسب:

سیاست SHiP وقتی بهترین عملکرد را دارد که پترن استفاده ی برنامه از یک سری خانه های حافظه مشخص و الگودار باشد؛ SHiP می‌تواند این الگوها را تشخیص بدهد و از آنها استفاده بکند (عملا استفاده های مجدد از یک سری از خانه های حافظه معنی دار و الگو دار باشند).

چون SHiP به مرور یاد می‌گیرد و جدول SHCT را باید پر کند، فرآیند یادگیری آن کمی طول می‌کشد و به همین دلیل در اوایل شاید خوب عمل نکند. همچنین وقتی که پترن و جنس الگوی دسترسی برنامه عوض می‌شود، چون جدول هنوز روی پترن قبلی یاد گرفته، اوایل خوب عمل نمی‌کند و بندری می‌زند.

نکات کد:

۲-۱۰-۱ - ساختار sampler و جدول SHCT

```
1 struct ship : public champsim::modules::replacement {
2     static constexpr int maxRRPV = 3;
3     static constexpr std::size_t SHCT_SIZE = 16384;
4     static constexpr unsigned SHCT_PRIME = 16381;
5     static constexpr unsigned SHCT_MAX = 7;
6
7     class SAMPLER_class
8     {
9     public:
10         bool valid = false;
11         bool used = false;
12         champsim::address address{};
13         champsim::address ip{};
14         uint64_t last_used = 0;
15     };
16
17     std::vector<SAMPLER_class> sampler;
18     std::vector<int> rrpv_values;
19     champsim::msl::categorizer<long> set_categorizer;
20     champsim::data::bits sampler_tag_bits;
21
22     std::vector<std::array<champsim::msl::fwcounter<champsim::msl::lg2(SHCT_MAX +
23         1)>, SHCT_SIZE>> SHCT;
```

ساختار داده SHCT یک جدول با ۱۶۳۸۴ خانه است که هر خانه ی آن یک شمارنده ۳ بیتی است (چون SHCT_MAX = 7 است).

هر خانه ی sampler یک بیت used دارد که می‌گوید آیا آن بلوک بعد از درج حداقل یک بار استفاده شد یا نه.

۲-۱۰-۲ - آموزش جدول از روی sampler

```
1 void ship::update_replacement_state(..., uint8_t hit)
2 {
3     // update sampler
4     if (set_categorizer.get_sample_category(set) == 0) {
5         auto s_idx = set / champsim::msl::get_sample_rate(NUM_SET);
6         auto s_set_begin = std::next(std::begin(sampler), s_idx * NUM_WAY + ...);
7         auto s_set_end = std::next(s_set_begin, NUM_WAY);
```

```

8
9     // check hit
10    auto match = std::find_if(s_set_begin, s_set_end, [addr = full_addr, shamt =
    sampler_tag_bits](auto x) {
11        return x.valid && x.address.slice_upper(shamt) == addr.slice_upper(shamt);
12    });
13    if (match != s_set_end) {
14        auto SHCT_idx = match->ip.slice_lower<32_b>().to<std::size_t>() % SHCT_PRIME;
15        SHCT[triggering_cpu][SHCT_idx] -= 1;
16        match->used = true;
17    } else {
18        match = std::min_element(s_set_begin, s_set_end, [](auto x, auto y) { return
    x.last_used < y.last_used; });
19
20        if (!match->used) {
21            auto SHCT_idx = match->ip.slice_lower<32_b>().to<std::size_t>() %
    SHCT_PRIME;
22            SHCT[triggering_cpu][SHCT_idx] += 1;
23        }
24
25        match->valid = true;
26        match->address = full_addr;
27        match->ip = ip;
28        match->used = false;
29    }
30
31    // update LRU state
32    match->last_used = access_count++;
33 }
34
35 if (hit)
36     get_rrpv(set, way) = 0;
37 }
    
```

در صورتی که hit داشته باشیم از شمارنده مربوط به آن signature یکی کم می شود و اگر بدون اینکه از بلوک اضافه شده استفاده می مجدد شود و evict شود شمارنده را یکی زیاد می کنیم.

بخش signature همان ۳۲ بیت پایین PC است که با SHCT_PRIME % روی جدول می می شود. بخش sampler برای خودش یک سیاست LRU مستقل دارد که با last_used و access_count پیاده شده و هیچ ربطی به RRPV های کش اصلی ندارد. یعنی sampler یک کش مجازی کوچک است که فقط برای آموزش نگه داشته می شود.

در صورت hit در cache مقدار RRPV را به ۰ ریست می کند.

۲-۱۰-۳ قطعه ۳ - استفاده از پیش بینی در زمان درج

```

1 void ship::replacement_cache_fill(uint32_t triggering_cpu, long set, long
  way, ...)
2 {
3     // handle writeback access
4     if (access_type{type} == access_type::WRITE) {
5         get_rrpv(set, way) = maxRRPV - 1;
6         return;
7     }
8
9     // SHIP prediction
10    auto SHCT_idx = ip.slice_lower<32_b>().to<std::size_t>() % SHCT_PRIME;
11
12    get_rrpv(set, way) = maxRRPV - 1;
13    if (SHCT[triggering_cpu][SHCT_idx].is_max())
14        get_rrpv(set, way) = maxRRPV;
15 }

```

همان بخش پویا که بر اساس مقدار جدول تصمیم می‌گیرد که مقدار اولیه RRPV چه باشد. حالت پیشفرض مثل SRRIP مقدار $\text{maxRRPV} - 1$ است و حالت خیلی بد مقدار maxRRPV را به خود می‌گیرد. شرط `is_max` همان بررسی حالت خیلی بد است که در صورتی که شمارنده پر شده باشد (یعنی مقدار ۱ را به خود گرفته باشد) فعال می‌شود.

۲-۱۰-۴ قطعه ۴ - انتخاب قربانی

```

1 long ship::find_victim(uint32_t triggering_cpu, uint64_t instr_id, long
  set, ...)
2 {
3     auto begin = std::next(std::begin(rrpv_values), set * NUM_WAY);
4     auto end = std::next(begin, NUM_WAY);
5
6     auto victim = std::max_element(begin, end);
7     if (auto rrpv_update = maxRRPV - *victim; rrpv_update != 0)
8         for (auto it = begin; it != end; ++it)
9             *it += rrpv_update;
10
11    return std::distance(begin, victim);
12 }

```

این تابع عیناً همان سیاست های SRRIP و DRRIP است.

مقایسه ی سیاست ها روی ردپا ها

برای تعیین عملکرد هر سیاست، از معیار IPC استفاده شده است. چرا که در نهایت، افزایش سرعت کلی برنامه را دقیق تر از معیار های دیگری نظیر Hit rate نشان می دهد.

۳-۱ سوال ۱: عملکرد میانگین هر سیاست

عملکرد میانگین هر سیاست در میان همه ی ردپا ها و همه ی تعداد set ها به این صورت است:

Average IPC/LRU	Average IPC	Strategy
1.0269	0.3799	bip
1.0244	0.3789	drrip
0.9962	0.3685	fifo
1.0000	0.3699	lru
0.9813	0.3630	mru
1.0057	0.3720	random
1.0292	0.3807	ship
1.0189	0.3769	srrip

جدول ۳-۱: میانگین IPC هر سیاست روی همه ی ردپا ها و همه ی اندازه های کش

که این مقادیر بسیار به هم نزدیک هستند. دلیل این اتفاق این است که تاثیر کش های لایه های بالاتر و باقی سخت افزار پردازنده بسیار بیشتر از تاثیر صرفا کش لایه ی آخر در سرعت کلی برنامه است. طبق این آمار، سیاست ship با بیشترین مقدار میانگین IPC، در میانگین همه ی ردپا ها بهترین عملکرد را داشته است.

۳-۲ سوال ۲: آیا یک سیاست ثابت در همه ی ردپا ها بهترین است؟

با توجه به نمودار های IPC، مثلا در ردپای mcf، سیاست های ship و srrip و bip بهترین عملکرد را دارند. ولی ship در ردپای gcc عملکرد بسیار پایینی دارد. همچنین srrip و bip در ردپای lbm و omnetpp عملکرد جالبی ندارند. پس می توان نتیجه گرفت هیچ سیاست ثابتی بین آنهایی که بررسی شدند وجود ندارد که در همه ی ردپا ها بهترین باشد.

۳-۳ سوال ۳: مقایسه ی سیاست های پیشرفته با LRU

با بررسی نمودار ها، می توان گفت در ردپای gcc، همه ی سیاست های پیشرفته (ship و srrip و drrip و bip) بهتر از lru عمل کرده اند. در مورد ردپای lbm می توان گفت که عملکرد سیاست های مختلف به شدت به تعداد set ها وابسته بوده. به طوری که در تعداد کمتر آن (حدود ۱۰۲۴ یا ۲۰۴۸ set) تفاوت معناداری بین سیاست های پیشرفته ی ship و srrip و drrip با lru وجود ندارد و صرفا سیاست bip بوده که بسیار بهتر از آن عمل کرده است. اما در تعداد set های بیشتر (مثلا ۴۰۹۶)، فقط عملکرد ship و drrip به طور قابل ملاحظه ای بیشتر از lru است. در مورد ردپای mcf، باز هم همه ی سیاست های پیشرفته از lru بهتر عمل کرده اند؛ صرفا drrip در تعداد set های پایین مقداری کمتر از بقیه عمل می کند. در نهایت در ردپای omnetpp می توان گفت که هیچ سیاست پیشرفته ای چندان بهتر از lru عمل نکرده است، مگر در تعداد set های پایینتر (حدود ۱۰۲۴) که باز هم تفاوت خیلی زیادی ندارند.

۳-۴ سوال ۴: کجا FIFO و LRU کفایت می کنند؟

با توجه به بخش قبلی و نیز نمودار ها می توان نتیجه گرفت که فقط در ردپای omnetpp، دو سیاست lru و fifo کفایت می کنند و عملکرد بسیار خوبی دارند.

۳-۵ سوال ۵: آیا کاهش Miss Rate همیشه به افزایش IPC منجر می شود؟

نه لزوما. اگرچه معمولا کاهش Miss Rate باعث افزایش IPC می شود، اما رابطه کاملا خطی نیست. برای مثال SHiP در بعضی ردپا ها نرخ Miss کمتری نسبت به LRU دارد ولی افزایش IPC آن بسیار اندک است. در بعضی موارد نیز اختلاف زیاد در Miss Rate تنها به افزایش جزئی IPC منجر شده است. دلیل آن این است که IPC علاوه بر کش، به عواملی مانند وابستگی های داده، پیش بینی انشعاب، اجرای موازی دستور ها، و سایر stall های پردازنده نیز وابسته است. بنابراین کاهش Miss Rate شرط لازم برای افزایش IPC است، اما همیشه شرط کافی نیست.

۳-۶ سوال ۴: رفتار سیاست های تطبیقی

سیاست هایی مانند BIP و DRRIP و SHiP تلاش می کنند رفتار برنامه را هنگام اجرا تشخیص داده و خود را با آن تطبیق دهند. در ردا هایی مانند mcf و lbm این سیاست ها توانسته اند خطوط مفید را مدت بیشتری در کش نگه دارند و از جایگزینی زود هنگام جلوگیری کنند؛ در نتیجه هم نرخ Miss کاهش یافته و هم IPC افزایش یافته است. در omnetpp، الگوی دسترسی از قبل با رفتار LRU سازگار بوده است؛ بنابراین سازوکار های تطبیقی سودی نداشته و حتی سربار تصمیم گیری باعث افت جزئی عملکرد می شود.

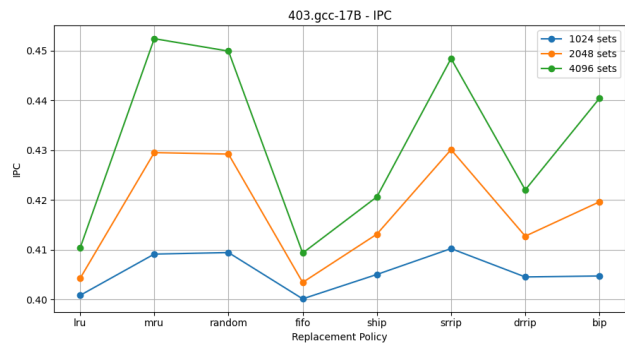
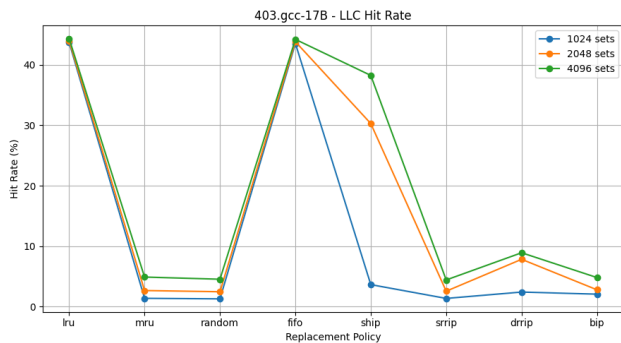
با بزرگ تر شدن اندازه ی کش (از ۱۰۲۴ به ۴۰۹۶ set)، مزیت سیاست های تطبیقی در ردا های ظرفیت محور مانند mcf بیشتر نمایان شده است، زیرا فضای بیشتر کش امکان استفاده ی بهتر از تصمیم های هوشمندانه را فراهم کرده است.

۳-۷ نمودار ها

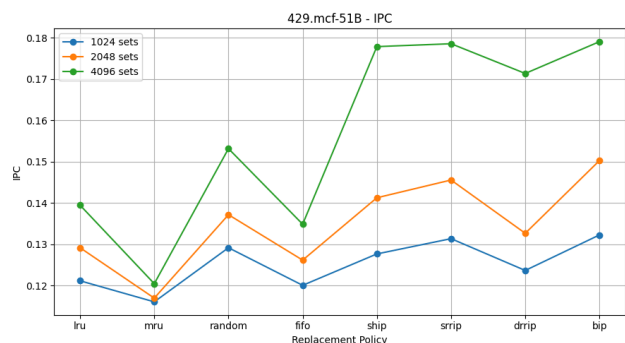
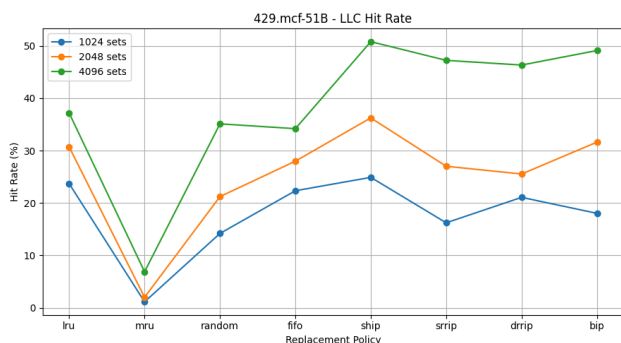
در این بخش تمام نمودار هایی که تحلیل های بالا بر اساس آنها نوشته شده اند را کنار هم آورده ایم. هر دو دسته هر سه اندازه ی کش (۱۰۲۴، ۲۰۴۸ و ۴۰۹۶ set) را پوشش می دهند و فقط در اینکه چه چیزی روی محور افقی است با هم فرق دارند.

۳-۷-۱ مقایسه بر حسب سیاست، برای هر سه اندازه ی کش

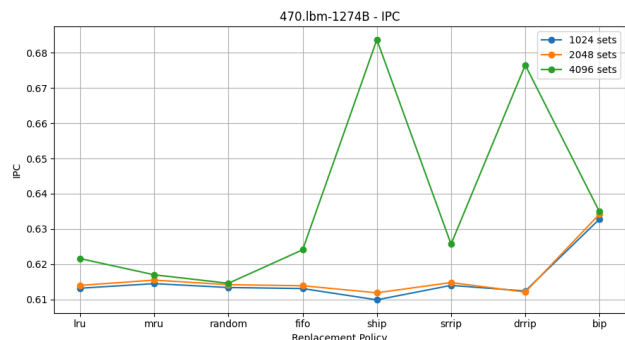
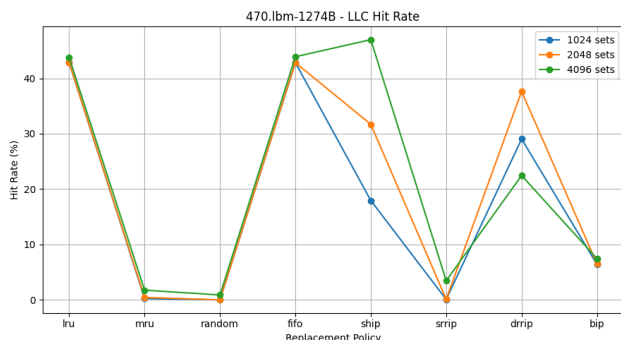
در این دسته محور افقی سیاست جایگزینی است و هر منحنی مربوط به یکی از سه اندازه ی کش (۱۰۲۴، ۲۰۴۸ و ۴۰۹۶ set) است، پس اثر خود سیاست را در اندازه های مختلف نشان می دهد.



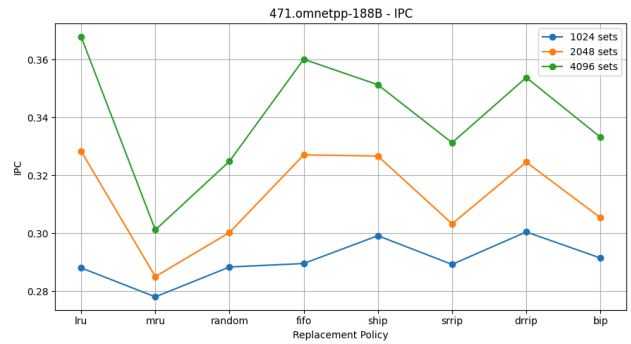
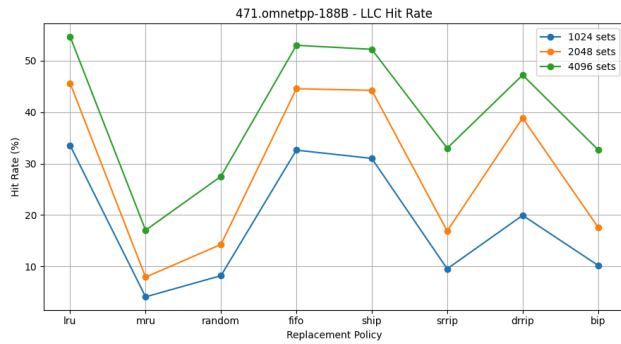
شکل ۳-۱: ردای IPC: 403.gcc و نرخ برخورد بر حسب سیاست، برای هر سه اندازه ی کش



شکل ۳-۲: ردای IPC: 429.mcf و نرخ برخورد بر حسب سیاست، برای هر سه اندازه ی کش



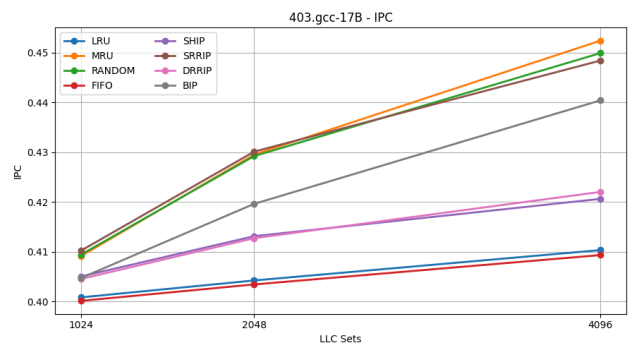
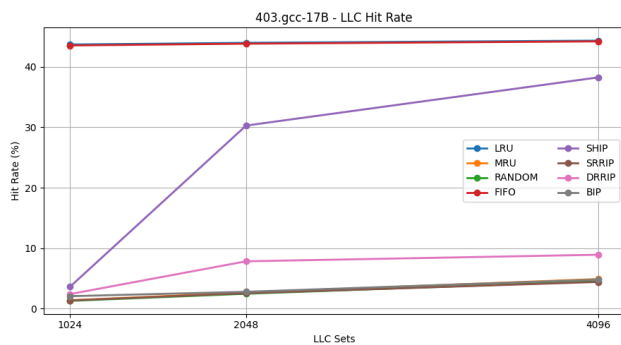
شکل ۳-۳: ردای IPC: 470.lbm و نرخ برخورد بر حسب سیاست، برای هر سه اندازه ی کش



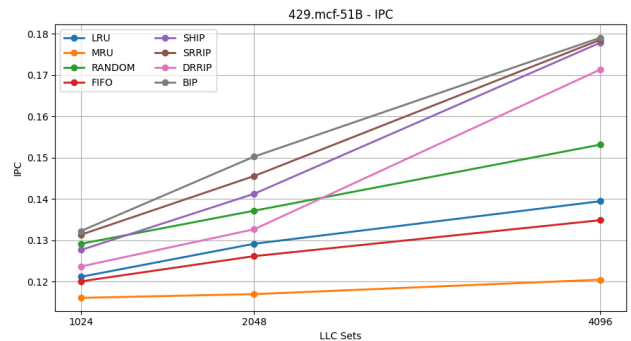
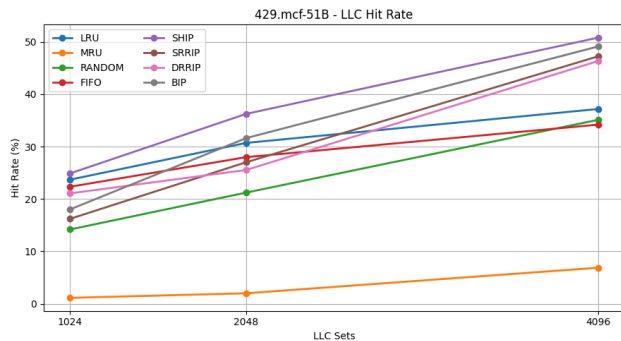
شکل ۳-۴: ردپای IPC 471.omnetpp و نرخ برخورد بر حسب سیاست، برای هر سه اندازه ی کش

۳-۱۱-۲ مقایسه بر حسب تعداد set

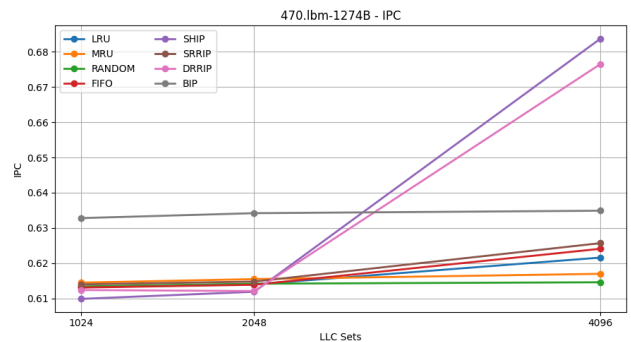
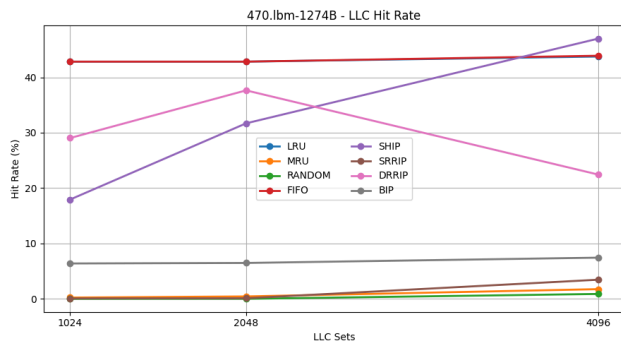
در این دسته محور افقی تعداد set هاست و هر منحنی مربوط به یک سیاست است، پس نشان می دهد هر سیاست با بزرگ تر شدن کش چطور رفتار می کند.



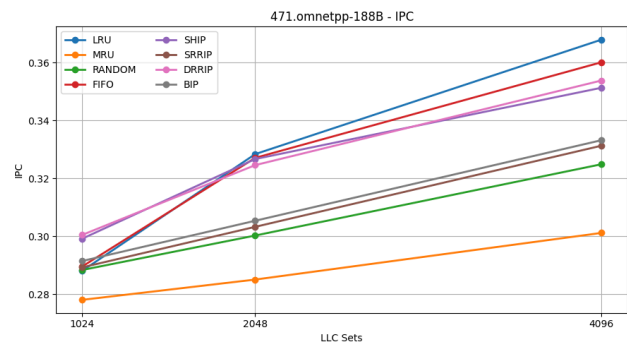
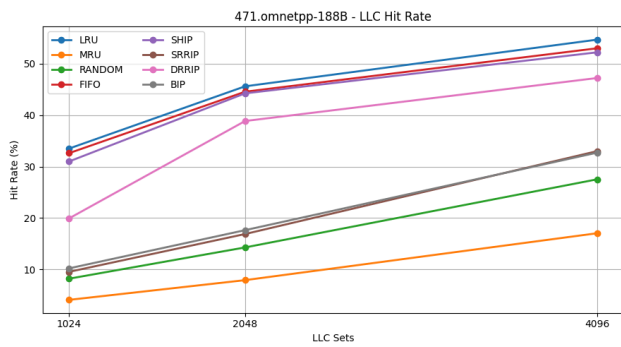
شکل ۳-۵: ردپای IPC 403.gcc و نرخ برخورد بر حسب تعداد set



شکل ۳-۶: ردپای IPC 429.mcf و نرخ برخورد بر حسب تعداد set



شکل ۳-۷: ردپای IPC 470.lbm و نرخ برخورد بر حسب تعداد set



شکل ۳-۸: ردپای IPC: 471.omnetpp و نرخ برخورد بر حسب تعداد set

توضیح Trace ها

در رابطه با trace ها انتخاب شده: trace اول gcc.403 بود و برای آن انتخابش کردیم که همجواری زمانی بالایی داشت. علت هم آن است که مربوط به کامپایلر gcc است کامپایلر بار ها به ساختار های داده یکسان (مثل جدول نماد ها و ...) دسترسی می‌خواهد بنابراین بلوک های کشی که به تازگی به از آنها استفاده شده، احتمالا دوباره از آنها استفاده خواهد شد و همانطور هم که در نمودار ها مشخص است و مورد انتظار ماست، سیاست LRU عملکرد خیلی خوبی روی آن داشته است.

ردپای دوم 429.mcf بود و علت انتخاب این بود که همجواری مکانی بالایی داشت. علت این اتفاق هم این است که این ردپا مربوط به حل مسایل بهینه سازی شبکه است و حجم خیلی زیادی داده را در حافظه پردازش میکند. این دسترسی ها معمولا به آدرس های مجاور هستند و برای همین همجواری مکانی در آن مهم است. یک نکته دیگر هم این است که به خاطر دسترسی های زیاد به حافظه، تاثیر اندازه کش در آن به خوبی محسوس است که در نمودار ها هم قابل مشاهده است.

ردپای سوم 470.lbm بود و علت انتخاب این بود که رفتار جریانی را درون خود دارد. این برنامه مربوط به شبیه سازی دینامیک سیالات است و داده ها را عمدتا به صورت ترتیبی و یک بار مصرف پردازش می کند. بنابراین پس از دسترسی به یک بلوک حافظه، احتمال استفاده مجدد از آن بسیار کم است. بنابراین این ردپا نمونه ی خوبی از رفتار جریانی محسوب می شود و همانطور که در نمودار ها واضح است hit rate برای برخی از سیاست ها بسیار پایین است که مطابق انتظار ماست.

ردپای آخر هم 471.omnetpp بود و برای این انتخاب شد که بین سیاست های جایگزینی تفاوت قابل قبولی قابل میشد. چرا که این ردپا مربوط به شبیه سازی شبکه های کامپیوتری است. الگوی دسترسی آن به نسبت پیچیده است و مجموعه داده های فعال آن به گونه ای است که انتخاب بلوک برای جایگزینی می تواند تاثیر قابل توجهی بر نرخ برخورد و عملکرد سیستم داشته باشد. این مورد هم دقیقا روی نمودار ها قابل مشاهده است و بین سیاست ها تفاوت وجود دارد.

نکته دیگر درباره شبیه سازی ها این است که تمام ۹۴ شبیه سازی، با ۱۲۵۰۰۰۰۰۰ دستور گرم سازی و ۵۰۰۰۰۰۰۰۰ دستور شبیه سازی انجام شد. این اتفاق خوبی بود که تمام شبیه سازی ها با تعداد دستور یکسان اجرا شدند زیرا باعث عدالت شد و تعداد دستورات هم مقدار زیادی بود تا به خوبی سیاست های مختلف روی ردپا های مختلف مورد آزمایش قرار بگیرند و تفاوت میان ردپا ها قابل حس باشد ولی طبیعتا به خاطر اینکه ردپا های مختلف داریم، تفاوت زمانی بین شبیه سازی ها دیده میشود. مثلا شبیه سازی ها با این تعداد دستور روی ردپای 429.mcf حدود ۲۰+ دقیقه زمان میبرد در حالی که بقیه شبیه سازی ها معمولا زیر ۱۰ دقیقه بودند و علت آن هم طبیعتا نوع متفاوت ردپاهاست و همانطور که گفته شده بود ردپای 429.mcf خیلی به حافظه دسترسی دارد (تعداد دسترسی ها همانطور که در llc_results مشاهده میشود برای این ردپا بیشتر از بقیه است) بنابراین نسبت به بقیه ردپا ها خیلی بیشتر در llc خطا داریم و از آنجا که خطا در llc پهنالتی خیلی زیادی دارد، باعث طولانی تر شدن شبیه سازی در این ردپا در مقایسه با بقیه شده است. میانگین دسترسی به llc در بقیه نزدیک به هم هستند و فقط 471.omnetpp مقدار کمی از بقیه کمتر به llc دسترسی دارد و همین مورد هم سبب این شده که این ردپا هم کمی نسبت به بقیه زمان کمتری بگیرد.

تحلیل پیچیدگی و هزینه ی سخت افزاری هر Policy

در گام های قبل سیاست ها را از نظر کارایی (نرخ برخورد و IPC) با هم مقایسه کردیم. اما در عمل، انتخاب یک سیاست جایگزینی فقط به کارایی آن وابسته نیست؛ هر سیاست برای پیاده سازی روی سخت افزار واقعی هزینه ای دارد که شامل بیت های کمکی ذخیره شده به ازای هر خط کش، نیاز به شمارنده های سراسری، و پیچیدگی منطق انتخاب قربانی و به روزرسانی وضعیت است. در این بخش هر هشت سیاست را از این جنبه ها بررسی می کنیم و در پایان جمع بندی می کنیم که با توجه به داده های به دست آمده، کدام سیاست از نظر نسبت کارایی به هزینه انتخاب بهتری است.

در پیاده سازی ما بسیاری از سیاست ها برای سادگی از یک برچسب زمانی پهن (uint64_t) استفاده می کنند؛ برای مثال LRU به ازای هر خط یک شمارنده ی زمانی ۶۴ بیتی نگه می دارد. این کار در یک شبیه ساز نرم افزاری کاملاً منطقی است، ولی در سخت افزار واقعی هیچ کس ۶۴ بیت به ازای هر خط خرج نمی کند. بنابراین در ستون بیت های کمکی ما هزینه ی سخت افزاری ایده آل (یعنی حداقل بیت لازم در یک پیاده سازی واقعی) را گزارش می کنیم و در صورت تفاوت، به پیاده سازی خودمان هم اشاره می کنیم. فرض ما یک کش ۱۶ راهه است (مطابق پیکربندی پایه). پس هر خط برای نگه داری ترتیب کامل به اندازه ی $\text{ceil}(\log_2(16)) = 4$ بیت نیاز دارد.

باید بین دو نوع شمارنده ی سراسری تفاوت قابل شویم. برخی سیاست ها مثل DRRIP به یک شمارنده ی سراسری الگوریتمی (مثل PSEL) نیاز دارند که بخشی از منطق تصمیم گیری سیاست است. اما در پیاده سازی نرم افزاری ما، سیاست های مبتنی بر برچسب زمان (LRU و FIFO و MRU و BIP) هم یک شمارنده ی cycle سراسری دارند که صرفاً برای تولید برچسب های زمانی افزایشی استفاده می شود. این شمارنده یک جز ذاتی الگوریتم نیست و در سخت افزار واقعی با روش های دیگری (مثل جابه جایی ترتیب در خود مجموعه) قابل حذف است. در جدول، این تفاوت را مشخص کرده ایم.

۵-۱ جدول مقایسه‌ی کلی

سیاست	بیت کمکی هر خط (ایده آل)	شمارنده‌ی سراسری	پیچیدگی انتخاب قربانی	پیچیدگی به روزرسانی	مناسب سخت افزار؟
Random	۰	ندارد (فقط مولد شبه تصادفی)	حداقلی: یک عدد تصادفی	ندارد	بله، ارزان ترین
FIFO	۴ بیت (ترتیب ورود)	فقط شمارنده‌ی زمان (نه الگوریتمی)	کم: کوچک ترین برچسب ورود	فقط هنگام درج (نه در hit)	بله
LRU	۴ بیت (ترتیب دسترسی)	فقط شمارنده‌ی زمان (نه الگوریتمی)	کم: کوچک ترین برچسب زمان	در هر hit و هر درج	نسبتاً؛ به روزرسانی پرهزینه
MRU	۴ بیت (مثل LRU)	فقط شمارنده‌ی زمان (نه الگوریتمی)	کم: بزرگ ترین برچسب زمان	مثل LRU	مثل LRU
SRRIP	۲ بیت (RRPV)	ندارد	متوسط: یافتن RRPV برابر ۳ و افزایش گروهی	مقدار ثابت در hit و درج	بله، بسیار مناسب
BIP	۴ بیت (مثل LRU)	شمارنده‌ی زمان + مولد تصادفی	مثل LRU	درج دوحالته (۱/۳۲)	بله
DRRIP	۲ بیت (RRPV)	شمارنده‌ی PSEL ۱۰ بیتی هر هسته	مثل SRRIP	SRRIP/BRRIP + به روزرسانی PSEL	بله، سربار کم
SHiP	۲ بیت RRPV + امضا هر خط	جدول SHCT با ۱۹۳۸۴ شمارنده‌ی ۳ بیتی + sampler	مثل SRRIP	جست و جو و به روزرسانی جدول SHCT	نسبتاً؛ پرهزینه ترین

جدول ۵-۱: مقایسه‌ی هزینه‌ی سخت‌افزاری و پیچیدگی هشت سیاست

۵-۲ بررسی سیاست به سیاست

۵-۲-۱ Random

این ارزان ترین سیاست ممکن است. هیچ اطلاعات کمکی به ازای هر خط ذخیره نمی‌کند (صفر بیت) و هیچ شمارنده‌ی سراسری‌ای ندارد؛ تنها به یک مولد شبه تصادفی نیاز دارد. انتخاب قربانی صرفاً تولید یک عدد در بازه‌ی $[0, \text{ways}-1]$ است و هیچ تابع به روزرسانی‌ای ندارد (در کد ما فقط `find_victim` نوشته شده است). از نظر سخت‌افزاری بی‌رقیب است، ولی چون هیچ اطلاعاتی از رفتار برنامه نگه نمی‌دارد، کارایی آن قابل پیش‌بینی نیست.

۵-۲-۲ FIFO

به ازای هر خط یک برچسب ترتیب ورود نگه می‌دارد (در پایه‌سازی ما `uint64_t`، ولی به طور ایده‌آل حدود ۴ بیت کافی است) و یک شمارنده‌ی زمان مشترک برای ثبت ترتیب ورود دارد. نکته‌ی مهم این است که تابع به روزرسانی آن خالی است؛ در هنگام hit هیچ کاری انجام نمی‌دهد و برچسب فقط در زمان درج ثبت می‌شود. به همین دلیل هزینه‌ی به روزرسانی آن از LRU کمتر است، ولی چون به دسترسی‌های مکرر بی‌توجه است، کارایی ضعیف‌تری دارد.

۵-۲-۳ LRU

به ازای هر خط یک برچسب زمان دسترسی نگه می‌دارد (ایده‌آل: ۴ بیت). شمارنده‌ی سراسری آن فقط برای تولید برچسب زمان است و جزئی از منطق تصمیم‌گیری نیست. انتخاب قربانی ساده است (کوچک‌ترین برچسب)، اما نکته‌ی اصلی هزینه‌ی آن در به روزرسانی است؛ در هر بار hit باید ترتیب دسترسی خط به روز شود. در سخت‌افزار واقعی، نگه‌داری ترتیب کامل LRU برای درجه‌ی هم‌نشینی بالا پرهزینه است و معمولاً از تقریب‌های آن (مثل Pseudo-LRU) استفاده می‌شود. به همین دلیل LRU با اینکه مبنای مقایسه‌ی ماست، لزوماً ارزان‌ترین گزینه نیست.

MRU ۵-۲-۴

از نظر سخت افزاری دقیقا هم هزینه ی LRU است؛ تنها تفاوت آن استفاده از max_element به جای min_element در انتخاب قربانی است. بنابراین همان ۴ بیت به ازای هر خط و همان منطق به روزرسانی را دارد. هزینه ی اضافه ای نسبت به LRU ندارد، ولی همان طور که در داده ها دیدیم در بیشتر ردپا ها کارایی بسیار پایینی دارد.

SRRIP ۵-۲-۵

از نظر نسبت کارایی به هزینه یکی از بهترین هاست. تنها ۲ بیت به ازای هر خط نیاز دارد (مقدار RRPV با $\max RRPV=3$) و هیچ شمارنده ی سراسری ای ندارد. این نکته جالب است که SRRIP هم کم بیت تر از LRU است (۲ بیت در برابر ۴ بیت) و هم در بیشتر ردپا ها کارایی بهتری دارد. انتخاب قربانی کمی پیچیده تر است چون ممکن است لازم باشد RRPV همه ی خطوط تا رسیدن یکی به مقدار بیشینه افزایش یابد (که در کد ما با یک transform جبری به جای حلقه پیاده شده تا ارزان تر باشد). ولی به روزرسانی آن بسیار ساده است: در hit مقدار صفر و در درج مقدار $\max RRPV-1$. از نظر سخت افزاری بسیار مناسب است.

BIP ۵-۲-۶

ساختار داده ی آن دقیقا مثل LRU است (۴ بیت به ازای هر خط) و انتخاب قربانی آن هم عینا مثل LRU است. تفاوت فقط در منطق درج است: با احتمال $1/32$ بلوک جدید را در موقعیت MRU و با احتمال $31/32$ در موقعیت LRU قرار می دهد. بنابراین سربار سخت افزاری اضافه ی آن تنها یک مولد تصادفی کوچک و پارامتر ϵ است. پیچیدگی آن کمی بیشتر از LRU خام است ولی همچنان کاملا مناسب سخت افزار است.

DRRIP ۵-۲-۷

روی SRRIP سوار شده و همان ۲ بیت RRPV به ازای هر خط را دارد. هزینه ی سراسری اضافه ی آن یک شمارنده ی ۱۰ بیتی به ازای هر هسته است (در کد ما $PSEL_WIDTH=10$) به همراه منطق دسته بندی مجموعه ها به سه گروه (رهبر SRRIP، رهبر BRRIP و follower). این هزینه ی سراسری بسیار ناچیز است؛ تنها ۱۰ بیت برای کل کش، در برابر ۲ بیت در هر تک تک خطوط. انتخاب قربانی مثل SRRIP است و به روزرسانی، علاوه بر منطق SRRIP / BRRIP، شامل به روز کردن PSEL در مجموعه های رهبر است. نکته ی جالب سخت افزاری این است که BRRIP پیاده سازی ما به جای احتمالی بودن، چرخه ای پیاده شده (هر ۳۲ درج یک بار). که از نظر عملکردی تقریبا یکسان ولی از نظر سخت افزاری ساده تر از یک مولد تصادفی است.

SHiP ۵-۲-۸

پرهزینه ترین سیاست از نظر سخت افزاری است. علاوه بر ۲ بیت RRPV به ازای هر خط، به یک امضا (signature) به ازای هر خط نیاز دارد (در کد ما مشتق شده از PC) و مهم تر از آن، یک جدول سراسری SHCT با 19384 خانه که هر خانه یک شمارنده ی ۳ بیتی است ($SHCT_MAX=7$). این جدول به تنهایی حجم قابل توجهی سخت افزار می طلبد. برای کاهش این هزینه، پیاده سازی به جای رصد همه ی مجموعه ها فقط تعداد محدودی مجموعه را به عنوان sampler رصد می کند و نتیجه را به بقیه تعمیم می دهد؛ ولی حتی با این ترفند، SHiP همچنان بیشترین بیت کمکی و پیچیده ترین منطق به روزرسانی (جست و جو، آموزش و به روزرسانی جدول) را دارد. انتخاب قربانی آن اما دقیقا مثل SRRIP ساده است؛ همه ی هزینه در منطق پیش بینی و به روزرسانی جدول است.

۵-۳ جمع بندی: کارایی در برابر سادگی

اگر تنها کارایی مهم باشد، انتخاب ما به اندازه ی کش بستگی دارد و این نکته ی جالبی است که در داده های ما دیده می شود:

- در کش های بزرگ تر (۴۰۹۶ مجموعه)، سیاست های پیشرفته ی تطبیقی بهترین عملکرد را دارند؛ به طور میانگین SHiP با بهبود حدود ۹ درصد و DRRIP با حدود ۸ درصد نسبت به LRU در صدر قرار می گیرند. در این حالت، هزینه ی سخت افزاری بالای SHiP توجیه می شود.
- در کش های کوچک تر (۱۰۲۴ و ۲۰۴۸ مجموعه)، جالب اینکه BIP با وجود سادگی، به طور میانگین بهترین بهبود (حدود ۴ درصد) را دارد و از SHiP و DRRIP هم جلو می زند.

بنابراین اگر صرفا کارایی مهم باشد و کش بزرگ باشد، SHiP یا DRRIP را انتخاب می کنیم.

اما اگر سادگی سخت افزاری هم مهم باشد، انتخاب ما تغییر می کند. در این حالت DRRIP گزینه ی بسیار جذابی است؛ تقریبا تمام قابلیت تطبیق پذیری SHiP را دارد ولی به جای یک جدول 19384 خانه ای و امضا به ازای هر خط، تنها به ۲ بیت در هر خط و یک شمارنده ی ۱۰ بیتی سراسری نیاز دارد. اگر بخواهیم از این هم ساده تر برویم، SRRIP با تنها ۲ بیت به ازای هر خط و بدون هیچ شمارنده ی سراسری، بخش بزرگی از بهبود را با کمترین هزینه ی ممکن به دست می آورد و حتی از LRU هم کم بیت تر است.

جمع بندی نهایی ما این است: DRRIP نقطه ی تعادل بهینه بین کارایی و هزینه ی سخت افزاری است. SHiP سقف کارایی را کمی بالاتر می برد ولی با هزینه ی سخت افزاری به مراتب بیشتر، و SRRIP ساده ترین انتخاب منطقی برای شرایطی است که بودجه ی سخت افزاری محدود است.

بخش ۴

نحوه ی اجرا و خروجی گرفتن پروژه

کل فرایند اجرا با سه اسکریپت داخل پوشه ی scripts انجام می شود که به ترتیب پشت سر هم اجرا می شوند: اول ساخت باینری ها، بعد اجرای شبیه سازی ها و در آخر استخراج نتایج از log ها. برای هر کانفیگ کش مجبوریم یک بیلد جدا بگیریم چون ChampSim اینطوری برنامه نویسی شده است.

۴-۱ اسکریپت make_bins.sh – ساخت باینری ها

این اسکریپت به ازای هر ترکیب از تعداد set و replacement policy یک باینری جدا می سازد. لیست مقادیر آن سه اندازه ی ۱۰۲۴ و ۲۰۴۸ و ۴۰۹۶ برای set و هشت سیاست lru و mru و random و fifo و ship و srrip و drrip و bip است و تعداد Way در همه ی حالت ها روی ۱۶ ثابت می ماند. در هر تکرار حلقه، با استفاده از qz سه فایلد LLC.sets و LLC.ways و LLC.replacement در فایل champsim_config.json بازنویسی می شود. بعد پوشه ی config.csconfig. پاک و با config.sh سورس ها دوباره تولید می شوند و در نهایت make اجرا می شود. باینری ساخته شده با نام گذاری <policy>w<ways>s<sets>llc_champsim داخل پوشه ی bin ذخیره می شود تا بعدا در نتایج قابل تشخیص باشد. در مجموع $۲۴ \times ۸ = ۳$ باینری ساخته می شود.

۴-۲ اسکریپت run_all.sh – اجرای شبیه سازی ها

این اسکریپت روی تمام باینری های داخل bin حلقه می زند و هر کدام را روی هر چهار ردپای انتخاب شده (403.gcc و 429.mcf و 470.lbm و 471.omnetpp) اجرا می کند، یعنی $۲۴ \times ۴ = ۹۶$ شبیه سازی که همان عددی است که در بخش ردپا ها به آن اشاره کردیم. تعداد دستورات هم برای همه یکسان و به صورت --warmup-instructions=12500000 و --simulation-instructions=50000000 تنظیم شده است. خروجی هر اجرا در فایلی با اسم log.<trace>_<binary> ذخیره می شود. برای اینکه اجرای کل مجموعه بیش از حد طول نکشد، شبیه سازی ها به صورت موازی و در پس زمینه اجرا می شوند و متغیر MAX_JOBS تعداد اجرا های همزمان را به N محدود می کند (مقدار معقول باید باشد که کل هسته های CPU درگیر نشوند که زمان اجرای آنها روی هم تاثیر بگذارد). حلقه ی انتظار داخل اسکریپت تا وقتی که تعداد job های در حال اجرا به زیر این حد نرسد اجرای جدید شروع نمی کند.

۴-۳ اسکریپت ex_info.sh – استخراج نتایج

بعد از تمام شدن شبیه سازی ها، این اسکریپت تمام فایل های داخل logs را می خواند و از هر کدام سه عدد مورد نیاز ما را بیرون می کشد: از خط LLC TOTAL cpu0->LLC مقادیر ACCESS و HIT و از آخرین خط cumulative IPC مقدار IPC نهایی. نتیجه به صورت یک فایل llc_results.csv با ستون های IPC, LLC_Hit, LLC_Access, Binary نوشته می شود که همان فایلی است که تمام نمودار ها و تحلیل های این گزارش از روی آن ساخته شده اند. نرخ برخورد هم از تقسیم LLC_Hit بر LLC_Access به دست می آید.

مراجع

- [1] Wikipedia, "Cache Replacement Policies." [Online]. Available: https://en.wikipedia.org/wiki/Cache_replacement_policies
- [2] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP)," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, pp. 60–71.
- [3] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive Insertion Policies for High Performance Caching," in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, pp. 381–391.
- [4] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, "SHiP: Signature-Based Hit Predictor for High Performance Caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2011, pp. 430–441.
- [5] N. Guber *et al.*, "The Championship Simulator: Architectural Simulation for Education and Competition," in *arXiv preprint arXiv:2210.14324*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.14324>
- [6] The 3rd Data Prefetching Championship (DPC-3), "DPC-3 ChampSim Trace Repository: SPEC CPU2006 Traces." [Online]. Available: <https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/>